



저작자표시 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.
- 이차적 저작물을 작성할 수 있습니다.
- 이 저작물을 영리 목적으로 이용할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#) 

**Incorporating Passage Importance in Retrieval-
Augmented Generation Models: A Study on
Passage Weighting Schemes**

Jaehoon Ha

**The Graduate School
Yonsei University
Department of Digital Analytics**

Incorporating Passage Importance in Retrieval-Augmented Generation Models: A Study on Passage Weighting Schemes

**A Master's Thesis Submitted
to the Department of Digital Analytics
and the Graduate School of Yonsei University
in partial fulfillment of the
requirements for the degree of
Master of Digital Analytics**

Jaehoon Ha

December 2024

**This certifies that the Master's Thesis
of Jaehoon Ha is approved**

Thesis Supervisor

Sang Yup Lee



Thesis Committee Member

Woo Ju Kim



Thesis Committee Member

Il Im



**The Graduate School
Yonsei University
December 2024**

TABLE OF CONTENTS

List of Figures		ii
List of Tables		iii
Abstract		iv
1. Introduction		1
2. Related Works		3
2.1. Generators		3
2.2. Retrievers		4
2.3. Fusion-in-Decoder		5
3. Methodologies		7
3.1. Weighting Methodologies		7
3.1.1. Constant Decrease		8
3.1.2. Decelerating Decrease		9
3.1.3. Accelerating Decrease		10
4. Experiments and Results		13
4.1. Datasets and Metric		13
4.2. Environments		13
4.3. Results		15
5. Conclusion		17
References		18
Abstract in Korean		20

List of Figures

<Figure 1> Architecture of Fusion-in-Decoder		5
<Figure 2> Architecture of Passage Weighted with Fusion-in-Decoder		7
<Figure 3> Graph of weights in CD method		9
<Figure 4> Graph of weights in DD method		10
<Figure 5> Graph of weights in AD method		11
<Figure 6> Visualization of Weight application		12
<Figure 7> Results of each Experiment		15

List of Tables

<Table 1> Weights of CD method		8
<Table 2> Weights of DD method		10
<Table 3> Weights of AD method		11
<Table 4> Experiment Environments		14
<Table 5> Experiment Hyperparameters		14
<Table 6> Model Combination		14
<Table 7> Results of each Experiment		15

ABSTRACT

Incorporating Passage Importance in Retrieval-Augmented Generation Models: A Study on Passage Weighting Schemes

Since the appearance of Transformer models, language models and large language models have pioneered implicit knowledge or parametric methodology that internalizes vast amounts of data through tens to hundreds of billions of parameters. However, this approach faces limitations, such as the high costs associated with integrating new information and updating data, as well as hallucination issues in untrained domains. To address these challenges, explicit knowledge or non-parametric methods that directly leverage external knowledge databases have emerged. While non-parametric methods allow for rapid information updates and corrections, mitigating hallucination issues, they also have limitations in their generative capabilities. To overcome these challenges, semi-parametric models have emerged, with Retrieval-Augmented Generation (RAG) marking a significant milestone. RAG has demonstrated exceptional performance across various natural language processing tasks. Subsequent research, including FiD (Fusion-in-Decoder) and Atlas, has expanded this paradigm; however, semi-parametric models still face limitations in effectively reflecting the importance of passages selected by the retriever. Therefore, this study proposes an innovative training approach based on the Fusion-in-Decoder (FiD) model, incorporating the rank of retriever-selected passages directly into the model's training process. By assigning higher weights to top-ranked passages, the model is designed to prioritize the most critical and relevant information. Ultimately, this training methodology aims to enhance performance over baseline models.

Key words: Large Language Model, Semi-Parametric, Retrieval-Augmented Generations, Fusion-in-Decoder, Passage Importance, Passage Weighting

1. Introduction

The core learning methodology of Transformer models, known as ‘Parametric’ or ‘Implicit Knowledge,’ adopts an approach where all data and information are embedded within the model's parameters[1]. While this methodology allows models to assimilate extensive knowledge from training data, it also reveals significant technical limitations. First, to incorporate new information or recent data, the model must be retrained, which requires enormous computational costs and time. Second, and more critically, there is the issue of hallucination. Hallucination occurs when the model confidently generates entirely fictitious information about untrained information, presenting it as if it were factual, leading to grave errors[2].

To overcome these limitations, the ‘Non-Parametric’ or ‘Explicit Knowledge’ has been proposed as an alternative. This approach leverages retrievers and readers to directly reference external databases or knowledge bases, allowing real-time updates of data[1]. However, the explicit knowledge methodology also has fundamental limitations. Specifically, its language generation capabilities are limited because it can only response based on the tokens contained within the passages retrieved by the retriever.

The first semi-parametric model, Retrieval-Augmented Generation[3] (RAG), emerged as an attempt to combine the strengths of both implicit and explicit knowledge approaches. RAG retrieves relevant information from external knowledge bases and integrates it into the language generation process, ensuring accuracy of information while maintaining language generation capabilities[4]. Therefore, semi-parametric models, which were the first to combine implicit and explicit methodologies, demonstrated excellent performance across various NLP tasks but still exhibited significant limitations. One of the key constraints of semi-parametric models is their equal treatment of retrieved passages in the generator. Among the passages retrieved by the retriever, some are inherently more relevant to the query. However, most semi-parametric models fail to adequately reflect these differences. For example, if the first retrieved passage has the highest relevance to the query,

it should not be treated with the same weight as the last passage during training.

To overcome these limitations, this research proposes a new training methodology based on the Fusion-in-Decoder[5] (FiD) model that directly and differentially incorporates the ranking and importance of passages into the model training process. By assigning higher weights to top-ranked passages, the proposed approach is designed to enable the model to focus more effectively on the most relevant information during training. Therefore, this study aims to enhance the performance of semi-parametric models while introducing a new training approach that assigns differential weights to passages based on their order of relevance.

2. Related Works

Research that was conducted to improve the performance of semi-parametric models primarily focused on two main components: the retriever and the generator[6]. This emphasizes the structural characteristics of semi-parametric and RAG models, where both components play critical roles in the answer generation process. Therefore, this section reviews the studies that have been conducted to enhance the performance of semi-parametric models, exploring advancements in both the generator and retriever components.

2.1. Generators

Research related to generators initially focused on maximizing the size of generative models, and subsequently proposed various generator learning methodologies to address the computational cost problems arising from using large-scale models. Representative models that have explored generator training methodologies include FiD[5], FiD-KD[7], FiE[8], and Atlas[9]. Among these, Facebook's FiD gained attention for its ability to efficiently process large volumes of passages by concatenating the representation vector of question and passage combinations after encoder operations to generate a single answer[5]. Following FiD, FiD-KD introduced knowledge distillation into the FiD structure, offering an improved model that reduces size and computational costs while minimizing performance degradation[7]. In contrast, the FiE model advanced efficiency by shifting the concatenation of question and passage representations to the very first stage of encoder, rather than at the end of the encoder process as in FiD[8]. The Atlas model, designed to integrate retrieval and generation in a unified training framework, stands out for its robust performance even with limited training data (few-shot learning)[9]. Particularly, FiD stands out as the first model to handle combinations of questions and multiple passages while significantly reducing computational costs and maintaining high performance[5]. As a result, it has been used as the baseline for various semi-parametric research, including FiD-KD, FiE, and Atlas.

2.2. Retrievers

Research on the retriever has focused on the assumption that the overall performance of a model is primarily determined by the retriever. Even if large language models are used in the reader or generator, the overall performance is limited if the retriever retrieves inappropriate passages or information[4]. This insight has driven the development of various techniques to address the issue, with key methods including Dual-Encoder[10], Reranker[11], DPR[12] (Dense Passage Retrieval), and Contriever[13]. Dual-Encoder map questions and passages to independent embedding spaces, then calculate their relevance using cosine similarity or dot product operations between the two vectors[10]. Reranker, on the other hand, break the passage selection process into two stages, where the retriever's initial search results are evaluated more finely, and the order of passages is reordered. Cross-encoder, rather than bi-encoder architectures are commonly used in Reranker, excelling in evaluating the relevance between questions and passages[11]. DPR is an end-to-end approach that uses BERT-based encoders to embed both queries and documents, calculating their similarity to select relevant passages. It utilizes dense vectors, enabling more clarified search processes[12]. DPR is widely used as the basic retriever in semi-parametric models, therefore, RAG and FiD both used DPR as their retriever to select passages. Finally, Contriever is a self-supervised retriever used in the Atlas model. It is designed to work effectively with unstructured data and leverages large amounts of unlabeled text data to learn the general relationship between questions and passages[13]. During training, it selects random sentence pairs from the same document for learning[9]. This method enables Contriever to be applied not only in question-answering tasks but also across a variety of natural language processing applications.

2.3. Fusion-in-Decoder

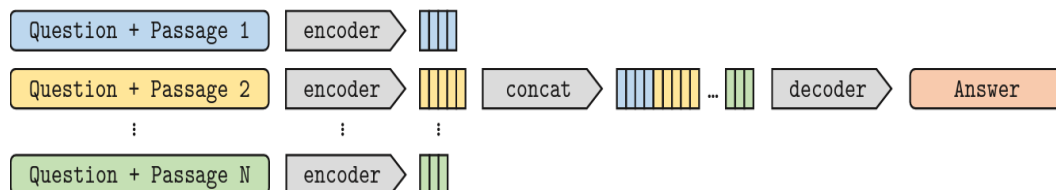


Figure 1. Architecture of Fusion-in-Decoder. This figure illustrates the architecture of the Fusion-in-Decoder method and is taken from the FiD paper[5].

This study proposes a new training methodology that applies differential weights based on the order of passages, using the FiD model as the baseline. Therefore, it is important to first understand the basic concept of the FiD model. FiD uses the same DPR retriever as RAG, however, it employs a different approach in the generator stage. FiD concatenates each question and passage, then converts them into a representation vector through the T5[14] encoder. Afterward, all the representation vectors of the questions and passages are concatenated into a single vector and passed to the T5 decoder to generate a single response[5]. The key difference from the RAG model is that FiD does not generate multiple answers, instead, it combines all passage information into one vector and passes it to the decoder to derive a single answer. This structure significantly reduces computational costs compared to RAG and provides higher efficiency, especially when processing many passages. RAG authors used up to 10 passages[3] in their experiments, however, FiD authors could handle up to 100 passages[5] because of its efficient cost reduction. Therefore, leveraging FiD as baseline for this study allowed experiments to be conducted using multiple passages even with a large generator model, without significant computational overhead.

The main objective of this study is to evaluate whether applying differential weights based on passage rankings can improve model performance. While recent research has focused on enhancing model capabilities through advancements in the core components of semi-parametric models, such as retrievers and generators, this research focuses on

improving performance without modifying the retriever and generator but with utilizing the FiD model and implementing a new training methodology that assigns differential weights according to passage ranking. In the original FiD model, passage ranking was just used to determine how many passages would be used for training. In contrast, this study proposes a new methodology where passage ranking is not just about the number of passages used but is applied to assign differential weights based on their order. This approach assures that the model prioritizes more relevant passages and better integrates their information. Therefore, this study uses the same retriever (DPR) and generator (T5) as the baseline FiD model but aims to improve baseline performance through a new training methodology that differentially weights passages based on their rank.

3. Methodology

The proposed methodology multiplies weights based on passage rankings to the representation vectors that were generated by the encoder operations for each question-passage pair. The passage ranked highest (passage 1) receives the largest weight, while passage 2 is assigned a slightly smaller weight, and passage 3 receives an even smaller weight and this process continues until all user-defined K passages have been weighed according to their rankings. Once these ranking-based weights are applied, the weighted representation vectors are concatenated into a single vector, which is then passed to the decoder. This design ensures that the decoder prioritizes information from higher-ranking passages, effectively integrating their contributions more prominently in the final response generation. This ranking-weighted training method optimizes the model's attention on the most relevant passages while still leveraging information from lower-ranked passages, improving the overall performance.

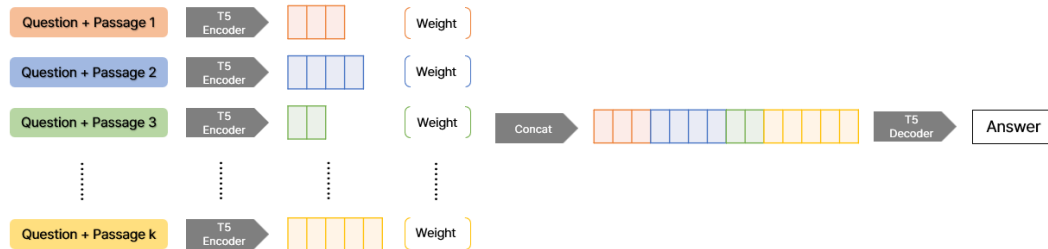


Figure 2. Architecture of Passage Weighted with Fusion-in-Decoder. This model, which applies weights to the baseline FiD, multiplies the appropriate weights corresponding to the passage order to the representation vectors of each question and passage after the encoder operations are completed. The weighted vectors are then concatenated and passed to the decoder.

3.1. Weighting Methodologies

This study proposes three different weighting methodologies. While one reason for conducting experiments with these three methods is to observe how performance varies

based on the weighting method, however, the primary goal is to accurately analyze whether assigning different weights based on passage order improves performance. The three proposed methods are: constant decrease, decelerating decrease, and accelerating decrease. All experiments were conducted using a maximum of 20 passages because of limitation of computational resources. Therefore, all the explanations provided in this paper are based on the 20-passage setup, however, the method is flexible, allowing users to adjust the number of passages (K) according to their specific computational constraints and needs.

3.1.1. Constant Decrease

$$W = 0.95^{k-1}$$

First, the constant decrease uses the powers of 0.95, ranging from the 0th power to the (k-1)th power, as set by the user. This approach results in gradually decreasing weight based on passage rank. The weights assigned to each passage rank when using 20 passages are shown in the table and figure below.

K	Weight	K	Weight
1	1	11	0.5987
2	0.95	12	0.5688
3	0.9025	13	0.5403
4	0.8573	14	0.5133
5	0.8145	15	0.4876
6	0.7737	16	0.4632
7	0.7350	17	0.4401
8	0.6983	18	0.4181
9	0.6634	19	0.3972
10	0.6302	20	0.3773

Table 1. Weights of CD method. This table shows the weights applied to each passage, from passage 1 to passage 20, when using the Constant Decrease method.

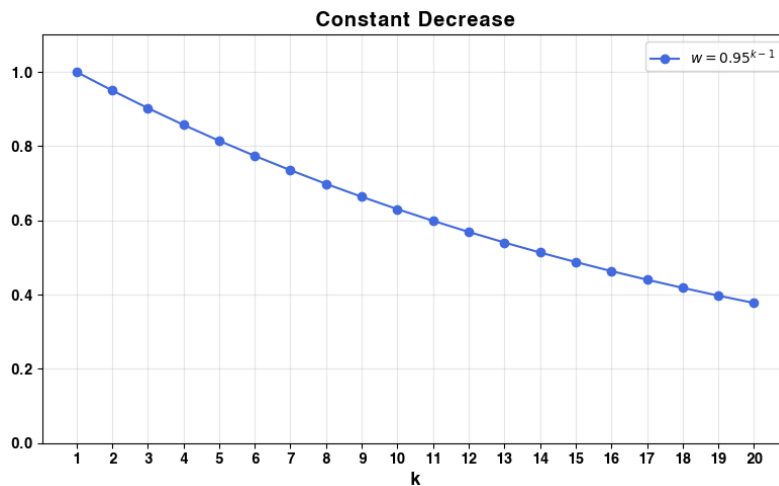


Figure 3. Graph of weights in CD method. The Constant Decrease weights are visualized in the figure as a graph.

3.1.2. Decelerating Decrease

$$W = \frac{1}{1+0.1 \cdot \ln(k)}, \quad (w = 1, \text{ when } k = 1)$$

The second method is the decelerating decrease using the natural logarithm function, where weights decrease rapidly at the beginning and then slow down towards the end. This method creates a significant difference in weights early on, while later weights become nearly equal. The weight for the first passage ($k=1$) is always set to 1. The weights assigned to each passage rank when using 20 passages are shown in the table and figure below.

K	Weight	K	Weight
1	1	11	0.8065
2	0.9351	12	0.8009
3	0.9010	13	0.7958
4	0.8782	14	0.7911
5	0.8613	15	0.7869
6	0.8480	16	0.7829
7	0.8371	17	0.7792
8	0.8278	18	0.7757
9	0.8198	19	0.7725
10	0.8128	20	0.7694

Table 2. Weights of DD method. This table shows the weights applied to each passage, from passage 1 to passage 20, when using the Decelerating Decrease method.

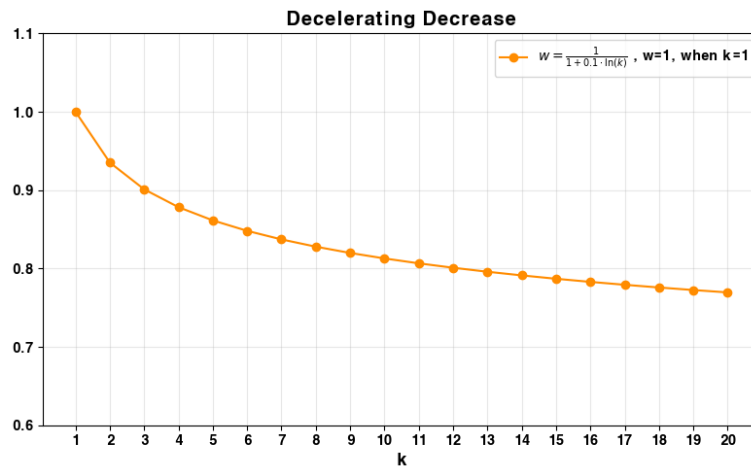


Figure 4. Graph of weights in DD method. The Decelerating Decrease weights are visualized in the figure as a graph.

3.1.3. Accelerating Decrease

$$W = e^{-0.001 \cdot (k-1)^2}$$

The final approach, the accelerating decrease methodology, utilizes the natural constant e to assign weights that decrease slowly at first but drop sharply for

lower-ranked passages. This method minimizes the difference in weights for higher-ranked passages while reducing the weights for lower-ranked ones. The weights assigned to each passage rank when using 20 passages are shown in the table and figure below.

K	Weight	K	Weight
1	1	11	0.9048
2	0.9990	12	0.8860
3	0.9960	13	0.8658
4	0.9910	14	0.8445
5	0.9841	15	0.8220
6	0.9753	16	0.7985
7	0.9646	17	0.7741
8	0.9521	18	0.7490
9	0.9380	19	0.7232
10	0.9221	20	0.6969

Table 3. Weights of AD method. This table shows the weights applied to each passage, from passage 1 to passage 20, when using the Accelerating Decrease method.

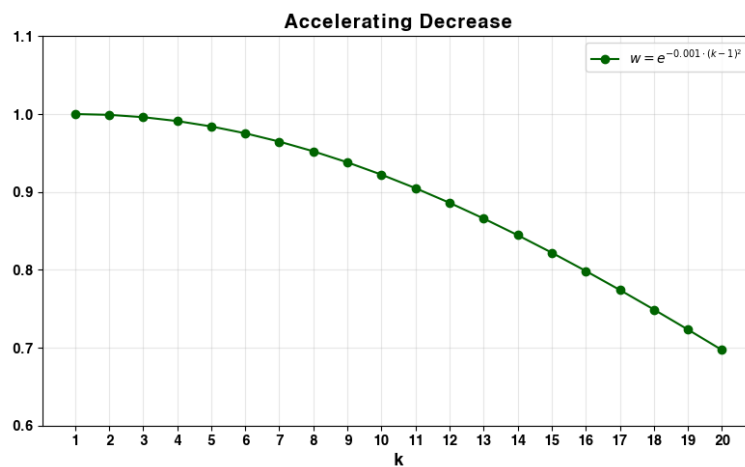


Figure 5. Graph of weights in AD method. The Accelerating Decrease weights are visualized in the figure as a graph.

Using FiD as the baseline, the study conducted training with three different weighting methodologies. These weights were appropriately multiplied after the encoder computation for each question-passage combination and then concatenated before being passed to the

decoder. By applying the three different weighting methods, the research aimed to precisely verify whether explicitly incorporating differential weights based on passage rank improves model performance. While how the weights are applied is important, this study aims to analyze whether the training methodology of assigning differential weights based on passage rank itself improves model performance.

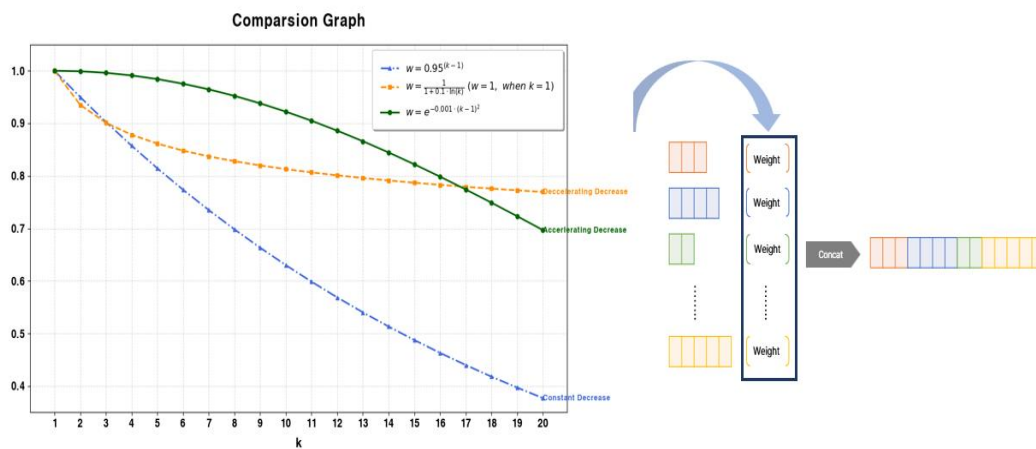


Figure 6. Visualization of Weight application. The weight values shown on the graph are appropriately multiplied with each question+passage representation, concatenated, and then passed to the decoder.

4. Experiments and Results

To ensure a fair comparison between the baseline model and the model that applies weights based on passage ranking, all experiments used DPR retriever[12] and T5[14]-base model for the generator. The FiD authors conducted experiments using the DPR retriever with the T5-large generator and up to 100 passages[5], however, due to GPU memory constraints, all experiments used a t5-base model and 20 passages. All experiments were conducted with the same environment, hyperparameters, datasets, and metrics to fairly compare baseline and weighted models.

4.1. Datasets and Metric

All experiments used the TriviaQA[15] and Natural Questions[16] from the open domain QA task to conduct a fair performance comparison with the FiD model. Two tasks, TriviaQA and Natural Questions, were also used in FiD paper[5], therefore, to fairly compare baseline with weighted models, all experiments were conducted using both tasks. All passages were also extracted using the Wikipedia dumps from Dec. 20, 2018, as used by the FiD authors, and both tasks were evaluated using Exact Match (EM) as the metric. To ensure consistency with the FiD authors, all experiments used the validation set as the test set for both TriviaQA and NQ tasks and used 10% of the training set as the validation set[5]. All experiments were conducted using the same training, dev, and test sets to fairly compare the results.

4.2. Environments

All experiments were conducted based on the experimental environments and hyperparameter tables provided below, and results were compared by sequentially conducting experiments according to the combinations in the table. Since this study could not use the same number of passages or the generator model(T5-large) as the FiD authors, a direct comparison with the results reported in the FiD paper was not possible.

Therefore, the FiD baseline model was also trained under the same experiment conditions as the models with applied weights, assuring clear result comparison.

Experiment Environments	
OS	CentOS Linux 6 (Core)
CPU	XEON#W-2135
GPU	Galaxy GeForce RTX 3090 SG D6X (24GB)
RAM	16GB * 8
HDD	SDD 512GB, HDD 8TB

Table 4. Experiment Environments. All experiments were conducted under the same conditions using the computational resources listed in this table.

Hyperparameters	
Generator	T5-base model[14]
# of Passages	20
Retriever	DPR[12]
Batch Size	2
Learning Rate	0.00005
Scheduler	Linear
Optimizer	AdamW[17]
Max_Length	250

Table 5. Experiment Hyperparameters. All experiments were conducted fairly using the same hyperparameters listed here.

Model Explanation	
FiD (baseline)	Fusion-in-Decoder
FiD w CD	Fusion-in-Decoder + Constant Decrease
FiD w DD	Fusion-in-Decoder + Decelerating Decrease
FiD w AD	Fusion-in-Decoder + Accelerating Decrease

Table 6. Model Combination. The results of the four model combinations were fairly compared by conducting each experiment individually.

4.3. Results

Model	TriviaQA	NQ
FiD	57.6	35.4
FiD + CD	58.69	36.54
FiD + DD	58.77	38.28
FiD + AD	58.73	38.03

Table 7. Results of each Experiment. This table shows the results of experiments on all models using the TriviaQA and NQ tasks. All weighted models, which were trained with an appropriate weight application, outperformed the baseline performance.

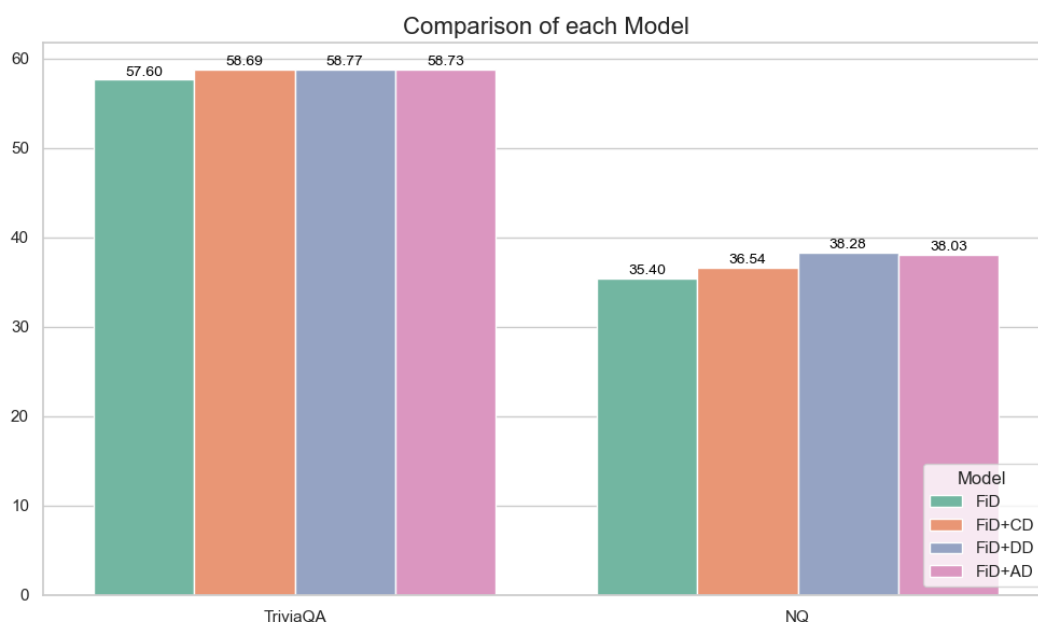


Figure 7. Results of each Experiment. This figure is a bar chart representing the results of all models. All scores were evaluated based on Exact Match (EM), and the FiD+CD, FiD+DD, and FiD+AD models with applied weights all outperformed the baseline performance.

As shown in the experimental results table, all three methodologies that applied differential weights according to passage order outperformed the FiD model in both tasks. First, applying the CD method to the FiD model improved EM scores by approximately 1.1 on both tasks. Also, applying the DD method, where weights decrease sharply at first

and taper off more gradually, yielded the best performance among all models, achieving a 2.9 EM improvement over the baseline in the Natural Questions task and a 1.2 EM improvement in the TriviaQA task. Finally, applying the AD method, where weights decrease gradually at first and then decrease sharply at the end, also produced better results than the baseline on both tasks.

The weighted models successfully improved baseline performance by applying differential weights according to passage order without modifying the generator and retriever. The proposed training approach allows the model to attend more on higher-ranked passages during training. The experimental results indicate that the formula used for weighting can be important, however, this study highlights that the methodology of assigning weights based on passage ranking itself is crucial for improving overall model performance. Furthermore, the experiments demonstrated that the approach of explicitly focusing the model more on higher-ranked passages, without increasing the number of parameters at all, had a positive impact on both tasks.

5. Conclusion

This study proposed three weighting methods—constant decrease, decelerating decrease, and accelerating decrease—to enhance the performance of the FiD model, one of the representative semi-parametric models, by assigning differential weights based on passage order. Without increasing the number of parameters and modifying the retriever and generator, the weight-based training methodology demonstrated overall performance improvements in the TriviaQA and NQ tasks. Due to constraints in the experimental environment, this study was unable to replicate the FiD authors' experiments using the T5-large model, process up to 100 passages, or optimize training with larger batch sizes. However, the finding underlines the practical contribution of explicitly assigning weights based on passage rankings to performance enhancement. Future research could further explore this approach by experimenting with more diverse tasks, exploring new weighting methodologies, and conducting additional research in environments with greater computational resources.

References

- [1] Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2022). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*.
- [2] Rawte, V., Chakraborty, S., Pathak, A., Sarkar, A., Tonmoy, S. M., Chadha, A., ... & Das, A. (2023). The troubling emergence of hallucination in large language models--an extensive definition, quantification, and prescriptive remediations. *arXiv preprint arXiv:2310.04988*.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [4] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- [5] Izacard, G., & Grave, E. (2020). Leveraging passage retrieval with generative models for open domain question answering. *arXiv preprint arXiv:2007.01282*.
- [6] Wang, M., Zhang, Y., Zhao, Q., Yang, J., & Zhang, H. (2024). Redefining Information Retrieval of Structured Database via Large Language Models. *arXiv preprint arXiv:2405.05508*.
- [7] Izacard, G., & Grave, E. (2020). Distilling knowledge from reader to retriever for question answering. *arXiv preprint arXiv:2012.04584*.
- [8] Kedia, A., Zaidi, M. A., & Lee, H. (2022). Fie: Building a global probability space by leveraging early fusion in encoder for open-domain question answering. *arXiv preprint arXiv:2211.10147*.
- [9] Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., ... & Grave, E. (2023). Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251), 1-43.
- [10] Dong, Z., Ni, J., Bikel, D. M., Alfonseca, E., Wang, Y., Qu, C., & Zitouni, I. (2022). Exploring dual encoder architectures for question answering. *arXiv preprint arXiv:2204.07120*.
- [11] Moreira, G. D. S. P., Ak, R., Schifferer, B., Xu, M., Osmulski, R., & Oldridge, E. (2024). Enhancing Q&A Text Retrieval with Ranking Models: Benchmarking, fine-tuning and deploying Rerankers for RAG. *arXiv preprint arXiv:2409.07691*.

- [12] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*.
- [13] Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E. (2021). Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*.
- [14] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140), 1-67.
- [15] Joshi, M., Choi, E., Weld, D. S., & Zettlemoyer, L. (2017). Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*.
- [16] Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., ... & Petrov, S. (2019). Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7, 453-466.
- [17] Loshchilov, I. (2017). Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

국문 요약

검색 증강 생성 모델에서 패시지 중요도 반영: 패시지 가중치 부여 기법 연구

트랜스포머 모델의 등장 이후, 언어 모형과 대규모 언어 모형은 수십억에서 수천억 개의 파라미터를 통해 방대한 데이터를 내재화하는 학습 방법론을 선도해왔다. 그러나 이러한 접근법은 근본적인 한계에 직면해 있다. 새로운 정보의 통합과 데이터 업데이트에 따르는 막대한 비용, 그리고 학습되지 않은 영역에서 발생하는 할루시네이션 문제가 대표적이다. 이러한 문제에 대응하여 외부 데이터베이스를 직접적으로 활용하는 명시적 지식 적용 방법론이 부상했다. 이 방법론은 정보의 신속한 업데이트와 수정을 가능케 하여 할루시네이션 문제를 완화했지만, 생성 능력의 제한이라는 또 다른 문제를 야기했다. 이러한 문제를 모두 해결하기 위해 등장한 세미 파라메트릭 모델의 시발점인, 검색 증강 생성(Retrieval-Augmented Generation, RAG)은 다양한 자연어 처리 작업에서 뛰어난 성능을 입증했다. 이후 FiD, Atals 등 후속 연구들이 이 패러다임을 확장해왔으나, 세미 파라메트릭 모델들은 리트리버가 선택한 패시지의 중요도를 제대로 반영하지 못하는 한계를 지니고 있다. 본 연구는 Fusion-in-Decoder(FiD) 모델을 기반으로, 리트리버가 선택한 패시지의 순위를 모델 학습 과정에 직접적으로 반영하는 혁신적인 학습 방법론 제안한다. 상위 패시지 일수록 더 높은 가중치를 부여함으로써, 모델이 가장 중요하고 관련성 높은 정보에 우선적으로 집중할 수 있도록 설계하였다. 궁극적으로 이 학습 방법론을 통해 베이스라인 모델보다 성능을 향상시키는 것을 목표로 한다.

핵심되는 말: 대규모 언어 모형, 세미 파라메트릭, 검색 증강 생성, 퓨전 인 디코더, 패시지 중요도, 패시지 가중치